

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL  
CUSCO

CARRERA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE  
SISTEMAS

## GUÍA DE LABORATORIO N.º 02

Algoritmos de aproximación aplicados a scheduling

List Scheduling y Longest Processing Time First

|                            |                                                           |
|----------------------------|-----------------------------------------------------------|
| <b>Asignatura:</b>         | Algoritmos Avanzados                                      |
| <b>Unidad:</b>             | I. Complejidad y algoritmos de aproximación               |
| <b>Semestre académico:</b> | 2026-II                                                   |
| <b>Duración:</b>           | 120 minutos                                               |
| <b>Organización:</b>       | Trabajo grupal en los equipos del proyecto semes-<br>tral |
| <b>Integrantes:</b>        | Aproximadamente cuatro estudiantes por equipo             |
| <b>Lenguaje sugerido:</b>  | Python 3                                                  |
| <b>Docente:</b>            | Héctor Eduardo Ugarte Rojas                               |

**Equipo:** \_\_\_\_\_

**Integrantes:** \_\_\_\_\_

**Fecha:** \_\_\_\_\_

# Índice

|                                                                        |          |
|------------------------------------------------------------------------|----------|
| <b>1. Objetivo de la práctica</b>                                      | <b>2</b> |
| <b>2. Trabajo preparatorio</b>                                         | <b>2</b> |
| <b>3. Materiales</b>                                                   | <b>3</b> |
| <b>4. Marco teórico breve</b>                                          | <b>3</b> |
| 4.1. Problema $P_m \parallel C_{\text{máx}}$ . . . . .                 | 3        |
| 4.2. Cota inferior . . . . .                                           | 3        |
| 4.3. List Scheduling y LPT . . . . .                                   | 3        |
| 4.4. Calidad observada . . . . .                                       | 3        |
| <b>5. Distribución sugerida de los 120 minutos</b>                     | <b>4</b> |
| <b>6. Ejercicios resueltos</b>                                         | <b>4</b> |
| 6.1. Ejercicio resuelto 1: representación y validación . . . . .       | 4        |
| 6.2. Ejercicio resuelto 2: List Scheduling y LPT . . . . .             | 5        |
| 6.3. Ejercicio resuelto 3: Ramificar-Podar exacto . . . . .            | 5        |
| <b>7. Ejercicios propuestos</b>                                        | <b>7</b> |
| 7.1. Ejercicio propuesto 1: evaluación de calidad . . . . .            | 7        |
| 7.2. Ejercicio propuesto 2: efecto del orden y escalabilidad . . . . . | 7        |
| <b>8. Producto de entrega</b>                                          | <b>8</b> |
| <b>9. Tabla de evaluación</b>                                          | <b>8</b> |
| <b>10. Referencias</b>                                                 | <b>9</b> |

## 1. Objetivo de la práctica

### Objetivo

Implementar y comparar algoritmos exactos y de aproximación para el problema de scheduling en máquinas paralelas idénticas,  $P_m \parallel C_{\max}$ . El equipo calculará cotas inferiores, verificará la factibilidad de las planificaciones y evaluará List Scheduling y LPT mediante makespan, razón de calidad observada, desviación relativa y tiempo de ejecución.

Al finalizar la práctica, el equipo será capaz de:

1. representar instancias, máquinas, trabajos y planificaciones;
2. implementar List Scheduling con una cola de prioridad;
3. implementar LPT como ordenamiento seguido de List Scheduling;
4. calcular y utilizar cotas inferiores del makespan;
5. calcular el óptimo para instancias pequeñas mediante Ramificar-Podar;
6. distinguir garantía teórica y calidad observada;
7. diseñar una comparación reproducible entre algoritmos.

## 2. Trabajo preparatorio

Antes del laboratorio, cada integrante deberá:

1. revisar la definición de  $P_m \parallel C_{\max}$ , carga de máquina y makespan;
2. revisar List Scheduling, LPT, cota inferior y razón de aproximación;
3. recordar el uso de una cola de prioridad mínima;
4. recordar los principios de Ramificar-Podar: solución incumbente, cota inferior y poda;
5. responder brevemente:
  - a) ¿Por qué List Scheduling es un algoritmo de aproximación y no solo una heurística?
  - b) ¿Cuál es la diferencia entre una garantía  $\rho$  y una razón observada  $r(I)$ ?
  - c) ¿Por qué alcanzar una cota inferior válida demuestra optimalidad?
  - d) ¿Por qué LPT procesa primero los trabajos de mayor duración?

### Preparación grupal

El código es grupal, pero cualquier integrante podrá ser seleccionado para explicar la representación, ejecutar una instancia, justificar una cota o modificar un algoritmo.

### 3. Materiales

- Computadora con Python 3.11 o posterior, recomendado.
- Editor o IDE y terminal.
- Biblioteca estándar: `heapq`, `time`, `random`, `statistics`, `csv` e `itertools`.
- Git y repositorio del equipo.
- Herramienta para generar tablas y gráficos.
- Instancias pequeñas para validación y medianas para experimentación.

### 4. Marco teórico breve

#### 4.1. Problema $P_m \parallel C_{\max}$

Se tienen  $n$  trabajos independientes con tiempos  $p_j > 0$  y  $m$  máquinas idénticas. Cada trabajo se asigna a una sola máquina, no se interrumpe y tarda lo mismo en cualquiera de ellas. Si  $L_i$  es la carga de la máquina  $i$ , el objetivo es minimizar:

$$C_{\max} = \max_{1 \leq i \leq m} L_i.$$

#### 4.2. Cota inferior

Para tiempos enteros se utilizará:

$$LB = \max \left\{ \left\lceil \frac{\sum_j p_j}{m} \right\rceil, \max_j p_j \right\} \leq \text{OPT}.$$

La primera parte proviene de la carga promedio y la segunda del trabajo más largo.

#### 4.3. List Scheduling y LPT

List Scheduling procesa los trabajos en el orden recibido y asigna cada uno a una máquina con carga mínima. Su garantía es:

$$C_{LS} \leq \left(2 - \frac{1}{m}\right) \text{OPT}.$$

LPT ordena primero los trabajos de mayor a menor y luego aplica List Scheduling. Su garantía es:

$$C_{LPT} \leq \left(\frac{4}{3} - \frac{1}{3m}\right) \text{OPT}.$$

#### 4.4. Calidad observada

Cuando se conoce el óptimo de una instancia  $I$ :

$$r(I) = \frac{\text{ALG}(I)}{\text{OPT}(I)}, \quad \delta(I) = \frac{\text{ALG}(I) - \text{OPT}(I)}{\text{OPT}(I)} \times 100\%.$$

La razón  $r(I)$  describe una instancia concreta; la garantía  $\rho$  se demuestra para todas las instancias.

## 5. Distribución sugerida de los 120 minutos

| Etapa       | Actividad                                            | Tiempo |
|-------------|------------------------------------------------------|--------|
| Inicio      | Organización y revisión del trabajo preparatorio     | 10 min |
| Resuelto 1  | Representación, validación, makespan y cota inferior | 18 min |
| Resuelto 2  | Implementación de List Scheduling y LPT              | 22 min |
| Resuelto 3  | Solución exacta mediante Ramificar-Podar             | 25 min |
| Propuesto 1 | Comparación de calidad sobre instancias pequeñas     | 20 min |
| Propuesto 2 | Escalabilidad y comportamiento del orden             | 15 min |
| Cierre      | Defensa individual y organización de evidencias      | 10 min |

## 6. Ejercicios resueltos

### 6.1. Ejercicio resuelto 1: representación y validación

Una planificación se representará como una lista de máquinas, y cada máquina contendrá los trabajos que le fueron asignados.

Listing 1: Makespan, factibilidad y cota inferior

```

1 from math import ceil
2 from collections import Counter
3
4
5 def cargas(planificacion):
6     return [sum(maquina) for maquina in planificacion]
7
8
9 def makespan(planificacion):
10    return max(cargas(planificacion), default=0)
11
12
13 def es_factible(trabajos, planificacion, m):
14     if len(planificacion) != m:
15         return False
16     asignados = [p for maquina in planificacion for p in maquina]
17     return Counter(asignados) == Counter(trabajos)
18
19
20 def cota_inferior(trabajos, m):
21     if m <= 0:
22         raise ValueError("El numero de maquinas debe ser positivo")
23     if not trabajos:
24         return 0
25     return max(ceil(sum(trabajos) / m), max(trabajos))
26
27
28 trabajos = [7, 6, 5, 4, 3]
29 planificacion = [[7], [6, 3], [5, 4]]
30
31 print(es_factible(trabajos, planificacion, 3))
32 print(cargas(planificacion))
33 print(makespan(planificacion))
34 print(cota_inferior(trabajos, 3))

```

La salida esperada es factible, tiene cargas (7, 9, 9), makespan 9 y cota inferior 9. Como la planificación alcanza la cota inferior, queda demostrado que es óptima.

### Multiplicidades

La validación utiliza `Counter` porque una instancia puede contener trabajos con la misma duración. Comparar conjuntos perdería las repeticiones.

## 6.2. Ejercicio resuelto 2: List Scheduling y LPT

Listing 2: Implementación mediante cola de prioridad

```
1 import heapq
2
3
4 def list_scheduling(trabajos, m):
5     if m <= 0:
6         raise ValueError("El numero de maquinas debe ser positivo")
7
8     # Tuplas: (carga, identificador, lista de trabajos)
9     heap = [(0, i, []) for i in range(m)]
10    heapq.heapify(heap)
11
12    for trabajo in trabajos:
13        carga, maquina, asignados = heapq.heappop(heap)
14        nuevos = asignados + [trabajo]
15        heapq.heappush(heap, (carga + trabajo, maquina, nuevos))
16
17    resultado = sorted(heap, key=lambda x: x[1])
18    planificacion = [asignados for _, _, asignados in resultado]
19    return planificacion, max(carga for carga, _, _ in resultado)
20
21
22 def lpt(trabajos, m):
23     ordenados = sorted(trabajos, reverse=True)
24     return list_scheduling(ordenados, m)
25
26
27 trabajos = [3, 7, 4, 6, 5]
28 print(list_scheduling(trabajos, 3))
29 print(lpt(trabajos, 3))
```

Ambos algoritmos obtienen makespan 9 en esta instancia. Esto no significa que sean algoritmos exactos; significa que, para esta instancia, la solución aproximada coincide con el óptimo.

### Regla de desempate

La tupla incluye el identificador de máquina. Así, cuando las cargas son iguales, la elección es determinista. Otras reglas de desempate también pueden producir soluciones válidas.

## 6.3. Ejercicio resuelto 3: Ramificar-Podar exacto

Para instancias pequeñas se calculará el óptimo. Los trabajos se ordenan de mayor a menor y LPT proporciona una primera solución incumbente.

Listing 3: Solución exacta mediante Ramificar-Podar

```
1 def branch_and_bound(trabajos, m):
2     trabajos = sorted(trabajos, reverse=True)
3     mejor_plan, mejor_valor = lpt(trabajos, m)
4     cargas_actuales = [0] * m
5     plan_actual = [[] for _ in range(m)]
6     nodos = 0
7
8     def buscar(k):
9         nonlocal mejor_plan, mejor_valor, nodos
10        nodos += 1
11
12        if k == len(trabajos):
13            valor = max(cargas_actuales, default=0)
14            if valor < mejor_valor:
15                mejor_valor = valor
16                mejor_plan = [x.copy() for x in plan_actual]
17            return
18
19        trabajo = trabajos[k]
20        cargas_probadas = set()
21
22        for i in range(m):
23            # Rompe simetrias entre maquinas de igual carga.
24            if cargas_actuales[i] in cargas_probadas:
25                continue
26            cargas_probadas.add(cargas_actuales[i])
27
28            nueva_carga = cargas_actuales[i] + trabajo
29            if nueva_carga >= mejor_valor:
30                continue
31
32            cargas_actuales[i] = nueva_carga
33            plan_actual[i].append(trabajo)
34
35            carga_total = sum(cargas_actuales) + sum(trabajos[k + 1:])
36            lb = max(max(cargas_actuales),
37                    ceil(carga_total / m),
38                    max(trabajos[k + 1:], default=0))
39
40            if lb < mejor_valor:
41                buscar(k + 1)
42
43            plan_actual[i].pop()
44            cargas_actuales[i] -= trabajo
45
46            # Si se probó una maquina vacia, las demas vacias son simetricas.
47            if nueva_carga == trabajo:
48                break
49
50            # Si la solucion LPT alcanza la cota global, ya es optima.
51            if mejor_valor == cota_inferior(trabajos, m):
52                return mejor_plan, mejor_valor, nodos
53
54        buscar(0)
55        return mejor_plan, mejor_valor, nodos
56
57
58 plan, optimo, nodos = branch_and_bound([7, 6, 5, 4, 3], 3)
59 print(plan, optimo, nodos)
```

En el ejemplo, LPT obtiene 9 y la cota inferior global es 9. El procedimiento certifica el óptimo sin desarrollar el árbol completo.

### Alcance

El algoritmo exacto es apropiado para instancias pequeñas. Su peor caso continúa siendo exponencial. El límite práctico debe determinarse experimentalmente sin bloquear el equipo.

## 7. Ejercicios propuestos

### 7.1. Ejercicio propuesto 1: evaluación de calidad

Genere o seleccione por lo menos 20 instancias pequeñas con  $m \in \{2, 3, 4\}$  y un tamaño que permita calcular el óptimo mediante Ramificar-Podar. Para cada instancia ejecute:

- List Scheduling con el orden generado;
- LPT;
- Ramificar-Podar.

#### Requisitos:

1. validar todas las planificaciones;
2. registrar  $LB$ ,  $OPT$ ,  $C_{LS}$  y  $C_{LPT}$ ;
3. calcular  $r_{LS}(I)$ ,  $r_{LPT}(I)$  y las desviaciones relativas;
4. registrar tiempos y nodos del método exacto;
5. identificar cuántas veces LS y LPT alcanzan el óptimo;
6. reportar promedio, mediana y máximo de las razones observadas;
7. comprobar que ninguna razón observada viola las garantías teóricas;
8. interpretar los resultados sin confundir prueba experimental y demostración.

**Pregunta de análisis.** ¿LPT domina a List Scheduling en todas las instancias evaluadas? Si encuentra empates o excepciones respecto de un orden particular, explíquelos.

### 7.2. Ejercicio propuesto 2: efecto del orden y escalabilidad

Trabaje con instancias medianas y grandes donde no sea necesario calcular el óptimo. Para cada conjunto de trabajos compare:

- el orden original;
- por lo menos 20 permutaciones aleatorias aplicadas a List Scheduling;
- el orden LPT.

#### Requisitos:

1. utilizar semillas reproducibles;

2. variar  $n$  y  $m$ ;
3. registrar makespan, cota inferior,  $C_{ALG}/LB$  y tiempo;
4. mostrar la distribución de makespan producida por distintos órdenes;
5. presentar al menos dos gráficos pertinentes;
6. explicar por qué  $C_{ALG}/LB$  no equivale necesariamente a  $C_{ALG}/OPT$ ;
7. identificar una instancia donde el orden afecte de manera visible a List Scheduling.

**Pregunta de análisis.** ¿El costo del ordenamiento de LPT resulta significativo frente a la mejora del makespan para los tamaños evaluados?

## 8. Producto de entrega

### Producto de entrega

El equipo entregará:

1. código fuente de List Scheduling, LPT y Ramificar-Podar (tanto ejercicios resueltos como propuestos);
2. informe de laboratorio con nombres de integrantes que contiene:
3. pruebas de factibilidad y casos límite;
4. generador o conjunto reproducible de instancias;
5. gráficos de calidad y rendimiento;
6. respuestas a las preguntas de análisis;
7. conclusiones grupales de 200 a 300 palabras.

## 9. Tabla de evaluación

| Criterio                                     | Indicador de logro                                                                                                                                                                  | Puntaje |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| Asistencia y participación en el laboratorio | Asiste, permanece durante la práctica y participa en implementación, pruebas, análisis y revisiones. Se determina individualmente.                                                  | 4       |
| Corrección y funcionamiento                  | List Scheduling, LPT y Ramificar-Podar funcionan; las planificaciones son válidas; las cotas, métricas y experimentos se calculan correctamente; se incluyen los casos solicitados. | 8       |
| Análisis de complejidad y resultados         | Justifica complejidades, interpreta calidad y rendimiento, diferencia garantía y observación, y reconoce límites del método exacto y de las cotas.                                  | 3       |
| Dominio individual                           | Explica los algoritmos, la cota y las métricas; predice resultados; ejecuta una instancia nueva o realiza una modificación breve.                                                   | 4       |

| Criterio | Indicador de logro | Puntaje |
|----------|--------------------|---------|
| Total    |                    | 19      |

## 10. Referencias

### Referencias

- [1] Vazirani, V. V. (2001). *Approximation Algorithms*. Springer.
- [2] Williamson, D. P. y Shmoys, D. B. (2011). *The Design of Approximation Algorithms*. Cambridge University Press.
- [3] Graham, R. L. (1969). Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2), 416–429.
- [4] Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to Algorithms* (4.<sup>a</sup> ed.). MIT Press.
- [5] Python Software Foundation. *Python 3 Documentation*. <https://docs.python.org/3/>